EdgeDTP: Efficient Vision Transformer with Dynamic Token Pruning for Edge Inference

1.1.1 Abstract

The deployment of Vision Transformers (ViTs) on resource-constrained edge devices is significantly hindered by their quadratic computational complexity with respect to token count. While static efficient architectures like MobileViT and EfficientFormer have made strides in reducing parameter counts, they often process informative and redundant image regions with equal computational expenditure. Conversely, dynamic token pruning methods such as DynamicViT demonstrate theoretical reductions in floating-point operations (FLOPs) but frequently fail to translate these savings into realized latency improvements on edge hardware due to unstructured memory access patterns and synchronization overheads. In this paper, we propose EdgeDTP, a latency-aware Dynamic Token Pruning framework specifically designed for edge inference. Unlike prior approaches that optimize solely for theoretical FLOPs, EdgeDTP incorporates a differentiable budget controller that learns to allocate token quotas based on layer-wise latency profiles. We introduce a lightweight, importance-based scoring module that identifies and prunes redundant background tokens early in the network, utilizing a soft-to-hard gating mechanism to maintain end-to-end differentiability during training. Furthermore, we present a rigorous evaluation protocol that avoids proxy metrics, focusing instead on real-device latency measurements on ARM-based mobile processors. Our methodology provides a pathway to bridge the gap between theoretical sparsity and practical on-device acceleration, enabling adaptive inference that scales computation according to input complexity without compromising model architecture stability.

Index Terms

Vision Transformers, Edge Inference, Dynamic Token Pruning, Model Compression, Deep Learning, Mobile AI, Neural Architecture Search, Computational Efficiency.

2. Introduction

The advent of the Vision Transformer (ViT) has revolutionized computer vision by introducing a paradigm shift from local convolutional processing to global self-attention mechanisms. ViTs have demonstrated superior performance over Convolutional Neural Networks (CNNs) in large-scale image classification, object detection, and segmentation tasks, primarily due to their ability to model long-range dependencies. However, this performance comes at a substantial computational cost. The self-attention mechanism, which is central to the transformer architecture, exhibits a computational complexity that is quadratic with respect to the number of input tokens. For a standard input resolution of 224x224, a ViT typically processes 196 patches (tokens), leading to significant memory bandwidth pressure and latency, particularly on edge

devices such as mobile phones, IoT sensors, and embedded controllers where power and thermal budgets are strictly limited.

The challenge of deploying ViTs on the edge is exacerbated by the "static" nature of standard transformer inference. In a traditional ViT, every image patch is processed through every layer of the network, regardless of its semantic importance. For instance, in an image of a bird flying against a clear blue sky, the vast majority of tokens represent the sky—a redundant background that contributes little to the classification decision after the initial layers. Yet, standard architectures like DeiT or Swin Transformers allocate equal computational resources to these background tokens as they do to the foreground object. This inefficiency is a critical bottleneck for edge deployment, where every millijoule of energy consumption matters.

Existing solutions to this efficiency problem broadly fall into two categories: static architectural optimizations and dynamic computation. Static approaches, such as MobileViT and EfficientFormer, redesign the transformer blocks to be more hardware-friendly, often by reintroducing convolutional biases or simplifying attention mechanisms. While effective, these methods still treat all spatial locations uniformly. Dynamic approaches, such as DynamicViT and Token Merging (ToMe), attempt to reduce the token count at runtime. However, a significant gap remains between the theoretical FLOPs reduction promised by these dynamic methods and the actual latency speedup observed on edge hardware. As noted in recent mobile inference studies, unstructured pruning or complex token merging operations can introduce memory copy overheads and index management costs that negate the benefits of reduced computation.

This paper addresses the gap by proposing a hardware-aware dynamic token pruning framework. We argue that for dynamic pruning to be effective on the edge, the pruning decisions must be guided not just by semantic importance, but also by the specific latency characteristics of the target hardware. We introduce EdgeDTP, which integrates a latency-aware budget controller into the training loop, ensuring that the token reduction schedule aligns with layer-wise processing costs.

The contributions of this work are as follows:

- We propose **EdgeDTP**, a dynamic token pruning framework that optimizes the trade-off between semantic information preservation and inference latency.
- We introduce a **Latency-Aware Budget Controller** that dynamically adjusts the pruning rate at different stages of the network based on a pre-computed latency lookup table for the target edge device, rather than relying solely on proxy FLOPs counts.
- We formulate a **Differentiable Pruning Mechanism** using a Gumbel-Softmax relaxation that allows for end-to-end training of the token selection module alongside the backbone network.
- We provide a rigorous **Evaluation Protocol** for measuring real-world edge latency, explicitly addressing the pitfalls of theoretical benchmarking and ensuring reproducibility in dynamic inference scenarios.

The remainder of this paper is organized as follows: Section 2 reviews related work in efficient vision transformers and dynamic computation. Section 3 details the proposed EdgeDTP method, including the importance scoring and budget control mechanisms. Section 4 discusses the implementation details specific to edge deployment. Section 5 outlines the evaluation protocol, and Section 6 discusses limitations and broader impacts. Finally, Section 7 concludes the paper.

3. Related Work

The pursuit of efficient deep learning models for computer vision has a rich history, evolving from lightweight Convolutional Neural Networks (CNNs) to the recent explosion of efficient Vision Transformer (ViT) architectures. This section reviews the landscape of efficient vision models, focusing on three key areas: static efficient transformers, dynamic computation mechanisms, and on-device inference considerations.

3.1 Vision Transformers and Efficient Variants

The original Vision Transformer (ViT) [1] demonstrated that pure attention-based architectures could outperform CNNs on large-scale datasets. However, ViT requires massive pre-training data and incurs high computational costs. DeiT (Data-efficient Image Transformers) [2] addressed the data efficiency issue through distillation and strong augmentation. More recently, DeiT III [13] revisited these supervised training recipes, showing that simplified yet stronger data augmentation can enable ViTs to match the performance of self-supervised methods without complex architectural changes. Despite these training improvements, the quadratic complexity of self-attention relative to token count remained a bottleneck.

To mitigate this, hierarchical architectures like Swin Transformer [9] introduced window-based attention to limit the computational scope. Similarly, Twins [14] proposed a spatially separable self-attention mechanism (SSSA) that alternates between local and global attention to balance efficiency with long-range context modeling. Hybrid architectures have also emerged to combine the strengths of CNNs and Transformers; for instance, MobileViT [5] combines MobileNet-style inverted residual blocks with transformer blocks. EfficientFormer [6] further revisits latency bottlenecks like reshaping operations, proposing a dimension-consistent design. While these static architectures reduce the baseline load, they lack input-adaptivity, processing simple and complex images with identical computational intensity.

3.2 Dynamic Computation and Token Compression

Dynamic neural networks adjust their structure or parameters based on the input sample. In the context of transformers, this often manifests as reducing the number of tokens as the data flows through the network. DynamicViT pioneered this direction by inserting "prediction modules" between transformer blocks to estimate the importance of each token. Tokens with low

importance scores are pruned (discarded), reducing the computational load for subsequent layers. This method showed that a significant portion of tokens (up to 66%) could be removed without substantial accuracy loss.

Alternative approaches focus on merging or adaptive computation. Token Merging (ToMe) [4] proposes a bipartite matching algorithm to combine similar tokens, reducing sequence length while preserving information. Addressing the information loss of hard pruning, SPViT [16] introduces a latency-aware soft pruning technique that "packages" less informative tokens into a representative token rather than discarding them entirely. In a similar vein of adaptivity, AdaViT [15] extends the Adaptive Computation Time (ACT) framework to vision transformers, automatically learning to halt processing for redundant spatial tokens. However, a significant gap often remains between theoretical reductions and realized speedup on edge hardware, a challenge our work specifically targets through hardware-aware budgeting.

3.3 On-Device and Edge Inference Considerations

Deploying transformers on edge devices (e.g., mobile phones, embedded systems) introduces unique constraints related to memory bandwidth, cache locality, and compiler optimization. A seminal study by Microsoft Research highlighted that theoretical FLOPs are a poor proxy for latency on mobile devices. The study found that memory access patterns and non-linear operations (like Softmax and GELU) often dominate execution time. Furthermore, unstructured sparsity—where tokens are dropped randomly based on content—can lead to thread divergence and poor utilization of SIMD instructions on CPUs and GPUs.

Consequently, for a dynamic pruning method to be practically useful on the edge, it must be "hardware-aware." It implies that the pruning overhead itself must be negligible, and the resulting tensor shapes must remain friendly to standard dense matrix multiplication kernels, or the system must employ specialized sparse kernels. Unlike previous works that focus largely on server-side GPU throughput or theoretical FLOPs, EdgeDTP is designed with these edge-specific constraints in mind, utilizing a budget controller that explicitly accounts for the latency profile of the target device.

4. Method

In this section, we present **EdgeDTP** (Edge-aware Dynamic Token Pruning), a framework designed to accelerate Vision Transformer inference on edge devices by selectively removing redundant tokens. Our approach is built upon a standard hierarchical Vision Transformer backbone (e.g., DeiT or ViT) and introduces three key components: a lightweight **Token Importance Predictor**, a **Differentiable Pruning Mechanism**, and a **Latency-Aware Budget Controller**.

4.1 Problem Definition and Notation

Let $\mathbf{X} \in \mathbb{R}^{N \times D}$ represent the sequence of input tokens at a given layer, where N is the number of tokens (patches) and D is the embedding dimension. In a standard ViT, N remains constant throughout the network (e.g., $N = 196$ for a 224x224 image with 16x16 patches). Our goal is to generate a binary mask $\mathbf{M} \in \{0, 1\}^N$ such that we retain only a subset of tokens $\mathbf{X}' \in \mathbb{R}^{K \times D}$, where $K < N$. The pruning decision should satisfy two conflicting objectives: maximizing the classification accuracy of the final model and minimizing the inference latency on the target edge device.

4.2 Token Importance Prediction

To determine which tokens to keep, we employ a lightweight prediction module inserted between transformer blocks. To minimize overhead, we utilize the class token (CLS) as a global context proxy. The predictor takes the current token embeddings $\mathbf{X}$ and the CLS token $\mathbf{x}_{cls}$ as input. For each patch token $\mathbf{x}_i$, we compute a local importance score π_i.

The prediction module consists of a simple Multi-Layer Perceptron (MLP) that projects the token features to a scalar score. To incorporate global context, we concatenate the patch embedding with the CLS token embedding before feeding it into the MLP:

$$\pi_i = \mathrm{MLP}(\mathrm{Concat}(\mathbf{x}_i, \mathbf{x}_{cls}))$$

This lightweight design ensures that the cost of computing importance scores does not outweigh the savings gained from pruning. The scores π represent the probability that a token is informative for the final classification task.

4.3 Differentiable Pruning via Gumbel-Softmax

Training a network with binary decisions (keep vs. drop) is non-trivial due to the non-differentiable nature of the discrete sampling operation. To enable end-to-end training, we employ the Gumbel-Softmax relaxation. Instead of hard thresholding, we sample the keep decision from a Gumbel distribution. During the forward pass, we generate a hard mask $\mathbf{M}$ to actually drop tokens and realize the computational savings. During the backward pass, we use the continuous approximation to allow gradients to flow back into the importance predictor MLP.

Specifically, we treat the decision for each token as a binary random variable. The decision vector $\mathbf{M}$ is computed as:

$$\mathbf{M} = \mathrm{StraightThrough}(\mathrm{GumbelSoftmax}(\pi))$$

This allows the network to learn which tokens are "important" by receiving gradients from the final classification loss. If dropping a token increases the classification error, the gradient will adjust the predictor to increase that token's importance score in future iterations.

4.4 Latency-Aware Budget Controller

A key innovation in EdgeDTP is the Latency-Aware Budget Controller. Most prior works use a fixed keep-rate (e.g., keep 70% of tokens) or optimize for FLOPs. However, removing 10 tokens in an early layer might save more latency than removing 10 tokens in a later layer due to differences in feature map dimensions and memory access patterns.

We formulate a latency budget loss $\mathcal{L}_{budget}$. Let $L_j(k)$ be the estimated latency of layer j when processing k tokens. We pre-compute a latency lookup table for the target edge device (e.g., Snapdragon 888 DSP or Apple Neural Engine) by profiling the transformer block with varying token counts. The budget loss forces the total estimated latency to meet a target T_{target} :

$$\mathcal{L}_{budget} = \left(\sum_j L_j(k_j) - T_{target} \right)^2$$

By optimizing this loss, the network learns to allocate the token budget across layers dynamically. It might learn to prune aggressively in early layers to save maximum latency, or preserve tokens until deep layers if the semantic information is critical.

4.5 Training Objective and Inference

The total training loss is a weighted sum of the cross-entropy classification loss, the distillation loss (following DeiT), and the budget loss:

$$\mathcal{L}_{total} = \mathcal{L}_{cls} + \lambda_{dist}\mathcal{L}_{distill} + \lambda_{budget}\mathcal{L}_{budget}$$

We adopt a teacher-student training paradigm where a pre-trained full-size ViT serves as the teacher. The distillation loss ensures that the pruned student network retains the feature representations of the teacher as much as possible, specifically aligning the CLS tokens.

The inference process is straightforward. At each pruning stage, the importance predictor computes scores, the top-k tokens are selected based on the learned budget, and the remaining tokens are discarded. The indices of kept tokens are tracked to potentially allow for visualization or structured output, although for pure classification, only the embeddings are needed.

Inference Pseudocode for EdgeDTP

```

def edge_dtp_inference(x, model, pruning_stages):
    # x: Input image tensor [Batch, 3, H, W]
    # pruning_stages: List of layer indices where pruning occurs

    tokens = model.patch_embed(x)
    cls_token = model.cls_token
    tokens = concat(cls_token, tokens)

    for i, layer in enumerate(model.blocks):
        # Normal transformer block forward
        tokens = layer(tokens)
```

```python

        # Check if this is a pruning stage
        if i in pruning_stages:
            # Separate CLS and patch tokens
            cls, patches = tokens[:, 0], tokens[:, 1:]

            # Predict importance scores
            scores = model.predictor[i](patches, cls)

            # Determine k (number of tokens to keep)
            # k can be fixed or dynamic based on confidence threshold
            k = model.get_budget(i)

            # Get indices of top-k scores
            top_k_indices = argsort(scores, descending=True)[:k]

            # Prune: Gather only the important tokens
            kept_patches = gather(patches, top_k_indices)

            # Reassemble
            tokens = concat(cls, kept_patches)

    return model.head(tokens[:, 0])

```

5. Implementation for Edge Inference

Bridging the gap between algorithmic pruning and hardware acceleration requires careful implementation. A naive implementation of token dropping in frameworks like PyTorch uses boolean masking (e.g., `tensor[mask]`), which triggers dynamic tensor shape operations. On edge AI accelerators and mobile GPUs, such dynamic shapes can disable critical compiler optimizations or force fallback to slower CPU execution.

5.1 Compiler and Runtime Considerations

For deployment, we target standard edge inference engines such as ONNX Runtime, TensorRT (for NVIDIA Jetson edge devices), and CoreML (for iOS devices).

- **Structured vs. Unstructured Pruning:** While our method conceptually supports unstructured pruning (dropping any token), this leads to irregular memory access. To mitigate this, we explore a "structured" variant where we prune tokens in contiguous blocks or enforce a fixed output size per stage during inference (e.g., always keep exactly 128 tokens at stage 1, 64 at stage 2). This allows the compiler to pre-allocate memory buffers of static sizes, avoiding runtime allocation overhead.

- **Index Management:** The "gather" operation required to select top-k tokens can be expensive. We implement this using optimized `top-k` kernels available in modern NPU libraries. By restricting pruning to only 2-3 specific stages in the network (rather than every layer), we amortize the cost of this reorganization.

5.2 Memory Layout and Batching

In batched inference (Batch Size > 1), dynamic pruning poses a challenge: different images in the same batch might have different numbers of "easy" vs. "hard" tokens. If we allow per-image token counts to vary, the resulting tensor becomes ragged (non-uniform shapes), which prevents efficient batching.

To address this for edge scenarios where batching is used (e.g., processing a video stream buffer), we enforce a **Batch-Uniform Pruning** strategy. We compute the importance scores for all images in the batch and select a global threshold, or more commonly, we enforce that all images in the batch are pruned to the same number of tokens K at each stage. While this might be sub-optimal for individual easy images, it guarantees a dense tensor structure $[B, K, D]$, enabling high-throughput SIMD execution.

6. Evaluation Protocol

To ensure the rigor and reproducibility of our results, we define a comprehensive evaluation protocol. We explicitly avoid reporting fabricated numbers in this proposal, but we detail the exact metrics, datasets, and baselines that will be used to validate the efficacy of EdgeDTP.

6.1 Datasets and Baselines

Dataset: Experiments are primarily conducted on the **ImageNet-1K** classification benchmark, the standard for comparing ViT architectures. For downstream transferability checks, we also utilize **COCO** for object detection, using the backbone as a feature extractor.

Baselines: We compare EdgeDTP against three categories of models:

1. **Static Baselines:** DeiT-Tiny/Small, MobileViT, and EfficientFormer. These represent the state-of-the-art in fixed-architecture efficiency.
2. **Dynamic Pruning Methods:** DynamicViT and Dynamic Transformers. Comparisons here isolate the impact of our latency-aware budget controller.
3. **Token Merging:** ToMe. We compare pruning against merging to analyze the trade-off between information preservation and gather-scatter overhead.

6.2 Metrics and Latency Measurement

We report the following metrics:

- **Top-1 Accuracy (%):** On the ImageNet validation set.

- **Throughput (images/sec) & Latency (ms):** Measured on real hardware. We strictly avoid using FLOPs as the primary efficiency metric.
- **Theoretical GFLOPs:** Reported only for reference to demonstrate the discrepancy between theoretical and realized speedup.

Hardware Setup: Latency is measured on a Qualcomm Snapdragon 888 (representative of high-end mobile) and an NVIDIA Jetson Nano (representative of edge IoT). We use the TFLite benchmark tool for CPU/DSP measurements and TensorRT for GPU measurements. We use a batch size of 1 for latency (simulating real-time interaction) and batch size 64 for throughput.

6.3 Experimental Plan (Templates)

The following tables outline how results will be presented.

Table 1: Comparison of EdgeDTP against static and dynamic baselines on ImageNet-1K. Note: Numbers are placeholders for protocol definition.

Model	Method Type	Params (M)	GFLOPs	Top-1 Acc (%)	Latency (Snapdragon 888) (ms)	Speedup vs Baseline
DeiT-Small	Static	-	-	-	-	1.0x
DynamicViT-S	Pruning (FLOP-aware)	-	-	-	-	-
EdgeDTP-S (Ours)	Pruning (Latency-aware)	-	-	-	-	-

Table 2: Ablation study on Budget Controller strategies.

Pruning Strategy	Keep Rate (Stage 1/2/3)	Top-1 Acc (%)	Latency (ms)
Fixed (Uniform)	0.7 / 0.7 / 0.7	-	-
Learned (FLOPs target)	Dynamic	-	-
Learned (Latency target)	Dynamic	-	-

7. Discussion

Trade-offs: We hypothesize that while EdgeDTP will achieve superior latency-accuracy trade-offs compared to unstructured pruning, it may struggle against highly optimized static architectures like MobileViT on extremely low-end hardware (e.g., microcontrollers) where any control flow overhead is prohibitive. The "latency-aware" budget might aggressively prune early layers to meet timing constraints, potentially discarding small objects that only become semantically resolved in deeper layers.

Threats to Validity: A major threat to validity in dynamic inference research is the variability of hardware states (thermal throttling, background processes) which can introduce noise into latency measurements. To mitigate this, our protocol requires "cold-start" and "sustained" latency measurements. Another limitation is the dependence on specific compiler versions (e.g., TensorRT 8.x vs 7.x), which can drastically change how "gather" operations are optimized.

Broader Impact: By enabling efficient ViT inference, EdgeDTP reduces the energy footprint of AI applications, contributing to "Green AI." However, making surveillance-capable models more efficient on edge devices also raises privacy concerns that must be addressed through responsible deployment guidelines.

8. Conclusion and Future Work

We presented EdgeDTP, a framework that bridges the gap between theoretical efficiency and practical edge latency for Vision Transformers. By integrating device-specific latency profiles directly into the training loop via a differentiable budget controller, EdgeDTP learns pruning policies that are truly hardware-friendly.

Future work will explore: (1) **Multi-task pruning**, where different tokens are kept for simultaneous classification and segmentation tasks; (2) **Hardware-Software Co-design**, optimizing the NPU hardware architecture itself to handle sparse token streams efficiently; and (3) **Hybrid approaches** that combine token merging in early layers (to preserve information) with pruning in later layers (for speed).

9. References

- [1] A. Dosovitskiy et al., "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale," in *ICLR*, 2021.
- [2] H. Touvron et al., "Training data-efficient image transformers & distillation through attention," *arXiv preprint arXiv:2012.12877*, 2021. doi: 10.48550/arXiv.2012.12877.
- [3] Y. Rao et al., "DynamicViT: Efficient Vision Transformers with Dynamic Token Sparsification," in *NeurIPS*, 2021.
- [4] D. Bolya et al., "Token Merging: Your ViT But Faster," in *ICLR*, 2023.

- [5] S. Mehta and M. Rastegari, "MobileViT: Light-weight, General-purpose, and Mobile-friendly Vision Transformer," in *ICLR*, 2022.
- [6] Y. Li et al., "EfficientFormer: Vision Transformers at MobileNet Speed," *arXiv preprint arXiv:2206.01191*, 2022. doi: 10.48550/arXiv.2206.01191.
- [7] X. Wang et al., "Towards Efficient Vision Transformer Inference: A First Study of Transformers on Mobile Devices," in *HotMobile*, 2022.
- [8] Y. Wang et al., "Not All Images are Worth 16x16 Words: Dynamic Transformers for Efficient Image Recognition," in *NeurIPS*, 2021.
- [9] Z. Liu et al., "Swin Transformer: Hierarchical Vision Transformer using Shifted Windows," in *ICCV*, 2021.
- [10] A. Howard et al., "Searching for MobileNetV3," in *ICCV*, 2019.
- [11] K. He et al., "Deep Residual Learning for Image Recognition," in *CVPR*, 2016.
- [12] K. He et al., "Masked Autoencoders Are Scalable Vision Learners," in *CVPR*, 2022.
- [13] H. Touvron et al., "DeiT III: Revenge of the ViT," *arXiv preprint arXiv:2204.07118*, 2022. doi: 10.48550/arXiv.2204.07118.
- [14] X. Chu et al., "Twins: Revisiting the Design of Spatial Attention in Vision Transformers," in *NeurIPS*, 2021.
- [15] H. Yin et al., "AdaViT: Adaptive Tokens for Efficient Vision Transformer," in *CVPR*, 2022.
- [16] Z. Kong et al., "SPViT: Enabling Faster Vision Transformers via Latency-aware Soft Token Pruning," in *ECCV*, 2022.
- [17] M. Sandler et al., "MobileNetV2: Inverted Residuals and Linear Bottlenecks," in *CVPR*, 2018.
- [18] O. Russakovsky et al., "ImageNet Large Scale Visual Recognition Challenge," *IJCV*, 2015.